10ª MARATONA DE PROGRAMAÇÃO DA FIPP



DIA 28/05/2014 - 14H

CADERNO DE PROBLEMAS

28/05/2014 – 14:00 às 17:30

Leia atentamente estas instruções antes de iniciar a resolução dos problemas. Este caderno é composto de 6 problemas, sendo que 2 deles estão descritos em inglês.

1. Não é permitido o uso de material impresso, livros, apostilas e dicionários. Apenas é permitido o uso de lápis, caneta ou lapiseira, borracha, régua e papel para rascunho (fornecido pela comissão organizadora). O acesso à internet é bloqueado durante a realização da prova. A ajuda do ambiente de desenvolvimento como C, C++ ou Java pode ser utilizada.
2. A correção das resoluções dos problemas será automatizada por meio do sistema de submissão eletrônica BOCA, baseada nos resultados obtidos com uma série de execuções dos algoritmos de cada equipe.
3. Siga atentamente as exigências da tarefa quanto ao formato da entrada e saída do seu algoritmo. Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.
4. Os problemas não estão ordenados, neste caderno, por ordem de dificuldade. Procure resolver primeiro os problemas mais fáceis.
5. Utilize os nomes indicados nos problemas para nomear os seus arquivos-fonte, de acordo com a linguagem de programação utilizada.
6. Os problemas devem ser resolvidos utilizando o raciocínio entrada-processamento-saída, ou seja, não é necessário armazenar toda a saída para exibi-la.

Observações:

Não utilize arquivos para entrada ou saída. Todos os dados devem ser lidos da entrada padrão (teclado) e escritos na saída padrão (tela). Utilize as funções padrão para entrada e saída de dados:

- em C: `scanf`, `getchar`, `printf`, `putchar`;
- em C++: as mesmas de C ou os objetos `cin` e `cout`;
- em Java:
```
Scanner sc = new Scanner(); int i = sc.nextInt();
String s = sc.next(); float f = sc.nextFloat();
System.out.println(); System.out.printf("%d",i);
```

Em C ou C++: use `sprintf(str,"%d",num);` em vez de `itoa(num,str,10);`

Faculdade de Informática de Presidente Prudente

http://www.unoeste.br/fipp/maratona

Cubos Coloridos (A – vermelha)

Arquivo fonte: *cubos.c*, *cubos.cpp* ou *cubos.java*

Crianças adoram brincar com pequenos cubos. Elas passam horas criando 'casas', 'prédios', etc. O irmãozinho de Tomaz acabou de ganhar um conjunto de blocos coloridos no seu aniversário. Cada face de cada cubo é de uma cor.

Como Tomaz é uma criança muito analítica, ele decidiu descobrir quantos "tipos" diferentes de cubos o seu irmãozinho ganhou. Você pode ajudá-lo? Dois cubos são considerados do mesmo *tipo* se for possível rotacionar um deles de forma que as cores nas faces respectivas dos dois blocos sejam iguais.

Entrada

A entrada contém vários casos de teste. A primeira linha do caso de teste contém um inteiro N especificando o número de cubos no conjunto ($1 \leq N \leq 1000$). As próximas $3 \times N$ linhas descrevem os cubos do conjunto. Na descrição as cores serão identificadas pelos números de 0 a 9. A descrição de cada cubo será dada em três linhas mostrando as cores das seis faces do cubo "aberto", no formato dado no exemplo abaixo. No exemplo abaixo, as faces do cubo têm cores de 1 a 6, a face com cor 1 está no lado oposto da face com a cor 3, e a face com cor 2 é vizinha das faces 1, 3, 4 e 6, e está no lado oposto da face com cor 5.

```
1
2 4 5 6
3
```

O final da entrada é indicado por $N = 0$.

A entrada deve ser lida da entrada padrão.

Saída

Para cada caso de teste seu programa deve imprimir uma linha contendo um único inteiro, correspondente ao número de tipos de cubos no conjunto dado.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	Saída para o exemplo de entrada
3 0 0 7 2 3 1 0 1 2 3 7 0 3 0 0 2 1 7 2 1 1 1 1 1 1 2 2 2 2 2 2 0	2 2

Campeonato de Futebol da FIPP (B – amarela)

Arquivo fonte: *futebol.c*, *futebol.cpp* ou *futebol.java*

Aproveitando que é o ano de Copa, e ainda Copa no Brasil, a FIPP resolveu organizar um campeonato de futebol, no qual podem participar tanto alunos, como professores e também ex-alunos. Regra importante: é proibido chutar a canela dos professores!

Todos os participantes foram divididos em N equipes para disputarem várias partidas, entretanto, duas equipes não poderiam jogar uma contra a outra mais de uma vez. Vence a equipe que fizer mais pontos.

O Prof. Moacir foi escolhido como responsável por registrar os gols de cada partida, por ser o integrante mais experiente da FIPP. Mas como ele era o mais sábio de todos, logo se cansou do campeonato e cochilou. Ao acordar, ele descobriu que o torneio havia acabado e as equipes estavam esperando para saber os resultados de todos os jogos.

Como o Prof. Moacir não quer que ninguém descubra que ele tirou um cochilo durante o campeonato e não tomou nota dos gols de cada partida, então decidiu recuperar os resultados de todos os jogos. Para isso, ele começou a perguntar o resultado para cada um dos times, só que descobriu que o verdadeiro vencedor do campeonato foi a amizade (empate geral), ou seja, cada equipe venceu as outras equipes exatamente K vezes. Sendo assim, ajude o prof. Moacir a obter a ordem cronológica do campeonato que atenda a todas as condições do campeonato ou, caso contrário, relate que não existe uma tabela de jogos possível com base nas condições apresentadas.

Entrada

A primeira linha contém dois inteiros N e K ($1 \leq N, K \leq 1000$), sendo N a quantidade de times e K a quantidade de vezes que cada time venceu uma partida.

O final da entrada é indicado por $N = K = 0$.

A entrada deve ser lida da entrada padrão.

Saída

Na primeira linha imprima um inteiro M número de jogos disputados. As M linhas seguintes devem conter as informações sobre todos os jogos, sendo um jogo por linha. A i-ésima linha deve conter dois inteiros Ai e Bi ($1 \leq Ai, Bi \leq N$; $Ai \neq Bi$). Os números Ai e Bi representam que no i-ésimo jogo a equipe com número Ai venceu a equipe com número Bi. Você pode assumir que as equipes são numeradas de 1 a N.

Se um campeonato não atender às condições do problema, então imprima –1.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	Saída para o exemplo de entrada
3 1 7 3 0 0	3 1 2 2 3 3 1 21 1 2 1 3 1 4 2 3 2 4 2 5 3 4 3 5 3 6 4 5 4 6 4 7 5 6 5 7 5 1 6 7 6 1 6 2 7 1 7 2 7 3

Restoring IPv6 (C – azul)

Source file: *ipv6.c*, *ipv6.cpp* or *ipv6.java*

An IPv6-address is a 128-bit number. For convenience, this number is recorded in blocks of 16 bits in hexadecimal record, the blocks are separated by colons — 8 blocks in total, each block has four hexadecimal digits. Here is an example of the correct record of a IPv6 address: "0124:5678:90ab:cdef:0124:5678:90ab:cdef". We'll call such format of recording an IPv6-address full.

Besides the full record of an IPv6 address there is a short record format. The record of an IPv6 address can be shortened by removing one or more leading zeroes at the beginning of each block. However, each block should contain at least one digit in the short format. For example, the leading zeroes can be removed like that: "a56f:00d3:0000:0124:0001:f19a:1000:0000" → "a56f:d3:0:0124:01:f19a:1000:00". There are more ways to shorten zeroes in this IPv6 address.

Some IPv6 addresses contain long sequences of zeroes. Continuous sequences of 16-bit zero blocks can be shortened to "::". A sequence can consist of one or several consecutive blocks, with all 16 bits equal to 0.

You can see examples of zero block shortenings below:

"a56f:00d3:0000:0124:0001:0000:0000:0000" → "a56f:00d3:0000:0124:0001::";

"a56f:0000:0000:0124:0001:0000:1234:0ff0" → "a56f::0124:0001:0000:1234:0ff0";

"a56f:0000:0000:0000:0001:0000:1234:0ff0" → "a56f:0000::0000:0001:0000:1234:0ff0";

"a56f:00d3:0000:0124:0001:0000:0000:0000" → "a56f:00d3:0000:0124:0001::0000";

"0000:0000:0000:0000:0000:0000:0000:0000" → "::".

It is not allowed to shorten zero blocks in the address more than once. This means that the short record can't contain the sequence of characters "::" more than once. Otherwise, it will sometimes be impossible to determine the number of zero blocks, each represented by a double colon.

The format of the record of the IPv6 address after removing the leading zeroes and shortening the zero blocks is called short.

You've got several short records of IPv6 addresses. Restore their full record.

Input

The first line contains a single integer n — the number of records to restore ($1 \le n \le 100$).

Each of the following n lines contains a string — the short IPv6 addresses. Each string only consists of string characters "0123456789abcdef:".

It is guaranteed that each short address is obtained by the way that is described in the statement from some full IPv6 address.

The input must be read from standard input.

Output

For each short IPv6 address from the input print its full record on a separate line. Print the full records for the short IPv6 addresses in the order, in which the short records follow in the input.

The output must be written to standard output.

Sample Input

```
6
a56f:d3:0:0124:01:f19a:1000:00
a56f:00d3:0000:0124:0001::
a56f::0124:0001:0000:1234:0ff0
a56f:0000::0000:0001:0000:1234:0ff0
::
0ea::4d:f4:6:0
```

Output for the sample Input

```
a56f:00d3:0000:0124:0001:f19a:1000:0000
a56f:00d3:0000:0124:0001:0000:0000:0000
a56f:0000:0000:0124:0001:0000:1234:0ff0
a56f:0000:0000:0000:0001:0000:1234:0ff0
0000:0000:0000:0000:0000:0000:0000:0000
00ea:0000:0000:0000:004d:00f4:0006:0000
```

IQ test (D – verde)

Source file: *iq.c*, *iq.cpp* or *iq.java*

Bob is preparing to pass IQ test. The most frequent task in this test is to find out which one of the given n numbers differs from the others. Bob observed that one number usually differs from the others in evenness. Help Bob — to check his answers, he needs a program that among the given n numbers finds one that is different in evenness.

Input

The first line contains integer n ($3 \le n \le 100$) — amount of numbers in the task. The second line contains n space-separated natural numbers, not exceeding 100. It is guaranteed, that exactly one of these numbers differs from the others in evenness. The last test case is followed by a line containing one zero.

The input must be read from standard input.

Output

Output index of number that differs from the others in evenness. Numbers are numbered from 1 in the input order.

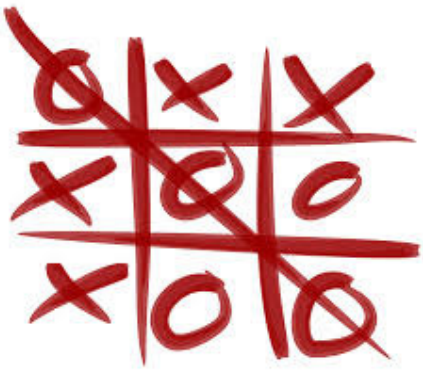
The output must be written to standard output.

Sample input	Output for the sample input
5 2 4 7 8 10 4 1 2 1 1 0	3 2

Jogo da Velha! (E – rosa)

Arquivo fonte: *velha.c*, *velha.cpp* ou *velha.java*

Certamente, todos estão familiarizados com o jogo da velha. As regras são muito simples de fato. Dois jogadores se revezam marcando as células em uma grade 3 × 3 (um jogador sempre desenha "X", o outro "0"). O jogador que conseguir colocar primeiro três de suas marcas em uma linha horizontal, vertical ou diagonal, ganha, e o jogo termina. O jogador que desenha "X" vai primeiro. Se a grade está cheia, mas nem Xs, nem 0s formam a linha requerida, um empate é anunciado. É dada uma grade 3 × 3, cada célula da grade está vazia, ou ocupada por um "X" ou um "0".



Você tem que encontrar o jogador (primeiro ou segundo), cuja vez é a próxima, ou imprimir um das sentenças a seguir:

ilegal — se o layout da grade dada não pode aparecer durante um jogo válido;

o primeiro jogador vence — se no layout da grade dada o primeiro jogador vence;

o segundo jogador vence — se no layout da grade dada o segundo jogador vence;

empate — se o layout da grade dada leva a um empate.

Entrada

A entrada consiste um inteiro *N* que indica a quantidade de rodadas do jogo que ocorreram. Para cada rodada, são lidas três linhas, cada uma das linhas contém os caracteres: ".", "X" ou "0" (um ponto, uma letra "X" maiúscula ou um dígito zero).

A entrada deve ser lida da entrada padrão.

Saída

Para cada rodada, imprima uma das seis sentenças: primeiro, segundo, ilegal, o primeiro jogador vence, o segundo jogador vence ou empate.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	Saída para o exemplo de entrada
3 X0X .0. .X. 0.X XX. 000 X0X 0X0 X..	segundo ilegal o primeiro jogador vence

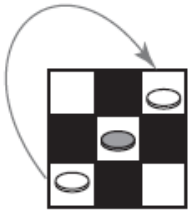
MegaDamas (F – branca)

Arquivo fonte: *damas.c*, *damas.cpp* ou *damas.java*

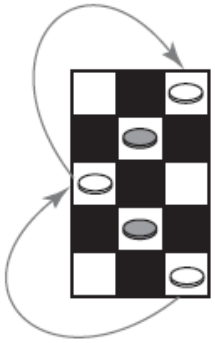
MegaDamas é um jogo de tabuleiro para dois jogadores, muito similar ao conhecido jogo de Damas. O tabuleiro é retangular, com N linhas e M colunas de pequenos quadrados arranjados em uma grade $N \times M$. Os pequenos quadrados são alternadamente coloridos com uma cor clara e uma cor escura, no padrão usual de um tabuleiro de damas. Os quadrados de cor escura são denominados "casas" (note, no entanto que, por razões de visualização, os diagramas abaixo mostram casas como quadrados brancos).

No início do jogo, cada jogador tem certo número de peças posicionadas nas casas mais próximas da borda do tabuleiro, que o jogador escolher (os jogadores escolhem bordas opostas). Durante o jogo, as peças só podem ocupar as casas brancas do tabuleiro.

Um dos movimentos do jogo é "capturar" uma peça do oponente, saltando sobre ela, diagonalmente, para a casa adjacente além da peça, casa esta que deve estar vazia. A peça do oponente é então removida do tabuleiro. As três casas envolvidas na captura (a casa inicial de sua peça, a casa que contém a peça do oponente e a casa vazia, onde sua peça estará após a jogada) devem estar diagonalmente alinhadas e devem ser diagonalmente adjacentes, como no diagrama abaixo.



Captura simples



Captura múltipla

Em MegaDamas uma peça pode capturar peças do oponente saltando diagonalmente para a frente ou para trás (note que, na maioria das variações existentes do jogos de Damas, uma peça só pode capturar peças oponentes saltando para a frente). Você pode também efetuar uma captura múltipla, com uma peça apenas, saltando seguidamente para casas vazias sobre peças oponentes. Em uma captura múltipla, a sua peça pode mudar de direção, saltando primeiro em uma direção e depois em outra. Você pode capturar apenas uma peça a cada salto, mas pode capturar várias peças com saltos seguidos. Você não pode saltar sobre uma peça sua, e não pode saltar a mesma peça oponente mais de uma vez.

São dadas as dimensões do tabuleiro e uma descrição do estado corrente de um jogo. É a sua vez de jogar e você deve determinar o número máximo de peças do seu oponente que podem ser capturadas em um movimento de captura.

Entrada

A entrada contém vários casos de teste. A primeira linha de um caso de teste contém dois inteiros N e M indicando respectivamente o número de linhas e o número de colunas do tabuleiro ($3 \leq N \leq 20$, $3 \leq M \leq 20$ e $N \times M \leq 200$). O quadrado mais à

esquerda do tabuleiro na borda mais próxima ao jogador é uma casa. A segunda linha contém a descrição do estado do jogo.

Cada descrição consiste de $\lceil (N \times M)/2 \rceil$ inteiros, separados por um espaço, correspondendo às casas do tabuleiro, que são numeradas de 1 a $\lceil (N \times M)/2 \rceil$, da esquerda para a direita, da borda mais próxima ao jogador à borda mais próxima ao seu oponente. Na descrição do estado do jogo, '0' representa uma casa vazia, '1' representa uma casa com uma de suas peças, e '2' representa uma casa com uma peça de seu oponente. Há no máximo $\lfloor (N \times M)/4 \rfloor$ peças de cada jogador no tabuleiro. O final da entrada é indicado por $N = M = 0$.

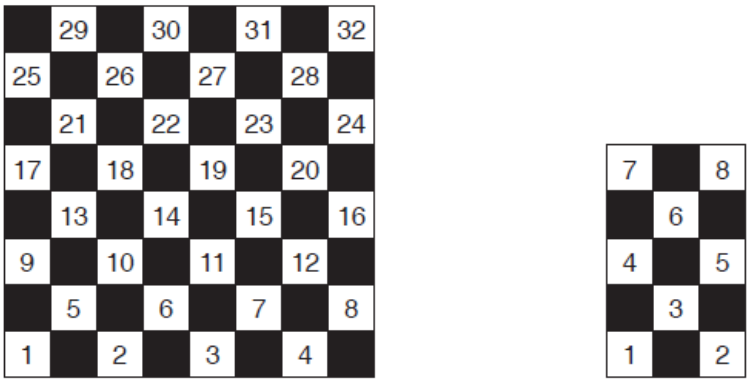


Figura 1: Numeração das casas em (a) tabuleiro de dimensões 8 × 8 e em (b) tabuleiro de dimensões 5 × 3.

A entrada deve ser lida da entrada padrão.

Saída

Para cada caso de teste da entrada, seu programa deve produzir uma única linha na saída, contendo um inteiro indicando o maior número de peças de seu oponente que podem ser capturadas em uma jogada.

A saída deve ser escrita na saída padrão.

Exemplo de entrada

```
3 3
2 1 2 0 1
5 3
1 0 2 1 0 2 0 0
8 8
2 2 2 2 0 0 0 0 2 2 2 2 0 0 0 0 2 2 2 2 0 0 0 0 2 2 2 2 0 1 0 0
0 0
```

Saída para o exemplo de entrada

```
1
2
7
```